

Machine Learning-Based Channel Prediction for RIS-Assisted MIMO Systems With Channel Aging

Sarthak Raj Swayam Kotecha Yash Khetan

Background & The Channel Aging Problem

Reconfigurable Intelligent Surfaces (RIS) in 6G

RIS technology functions as a smart reflection hub that manipulates electromagnetic propagation to improve signal quality. However, estimating the full channel matrix for hundreds of passive RIS elements requires transmitting massive amounts of pilot signals, consuming valuable time and frequency resources.

The Phenomenon of Channel Aging

In real-world scenarios, users are highly mobile. This mobility induces Doppler shifts, causing the wireless channel to vary rapidly over time. Consequently, by the time a complex RIS channel estimation finishes, the acquired Channel State Information (CSI) is frequently already outdated—a critical bottleneck known as channel aging.

Limitations of Existing Methods & Our Objective

Gaps in the Current Literature

- **High Overhead:** Traditional sequential ON/OFF or DFT-based estimation methods incur unbearable pilot overhead as RIS elements scale.
- **Structural Mismatch:** Existing Machine Learning (ML) channel predictors were built for standard MIMO and fail to account for the unique, cascaded structure ($BS \rightarrow RIS \rightarrow UE$) of an RIS-assisted environment.

Our Objective

Develop a specialized machine learning framework that replaces repeated estimation with intelligent prediction. This approach aims to drastically reduce pilot overhead, maintain accurate CSI despite aging, and improve overall spectral efficiency.

System Model: Network Architecture

The Physical Setup

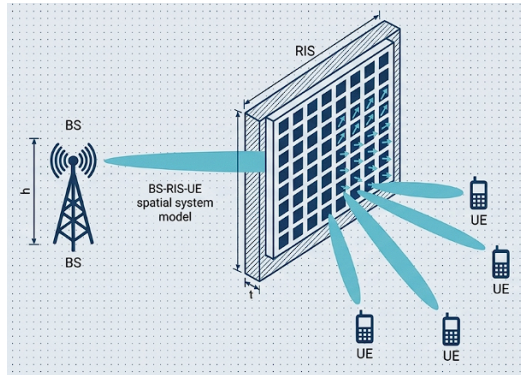
We model a TDD-based Multi-User MIMO system where a Base Station (BS) with N antennas communicates with K single-antenna users, assisted by an RIS. To simplify, the $\tilde{M}$ reflecting elements are grouped into M highly correlated sub-surfaces.

Cascaded Channel Equation

Signals travel via a direct BS-UE path ($\mathbf{d}_k$) and a reflected BS-RIS-UE path. The effective channel for the k^{th} user is mathematically represented as:

$$h_k[l] = G_k[l]\theta[l] + d_k[l]$$

Where G_k is the cascaded channel matrix and θ is the RIS phase shift vector.



System Model: The AR Baseline & Challenges

Issues with AR Alone

The AR model predicts the current channel from past measurements. However, it only works well when the order (Q) is high. A high Q requires massive amounts of past CSI data, making it computationally expensive and hard to implement in practice.

The RIS Complexity Issue

An RIS channel equals a static BS-RIS link combined with time-varying UE parts. This mixed nature makes the aging pattern exceptionally hard for a standard AR model to learn on its own.

Our Solution

Use a CNN to learn the complex aging pattern, then use the AR model to execute the prediction.

Autocorrelation Function (ACF)

Channel memory decays based on user velocity:

$$R[l] = J_0(2\pi f_n |l|)$$

Diagonal Loading Stabilization

Fixes matrix crashes for high Q values:

$$\hat{R}[q] = R[0] + \epsilon \quad (\text{for } q = 0)$$

System Modeling: Path Loss Model (Important for Results)

Path Loss Equation

The large-scale fading for the signal propagation is modeled as:

$$L(d) = \frac{L_0}{d^\alpha}$$

- $L_0 = -30$ dB (The reference loss at 1m)
- d = distance
- α = path loss exponent

Values Used:

- **BS-UE:** $\alpha = 3$
- **RIS-UE:** $\alpha = 3$
- **BS-RIS:** $\alpha = 2$

Important Insight

The BS-RIS link has a lower path loss exponent ($\alpha = 2$), meaning the **RIS link is less lossy**. This validates why placing an RIS improves signal quality compared to relying on direct paths alone.

Channel Estimation & TDD Framework

Signal Modeling & Estimation

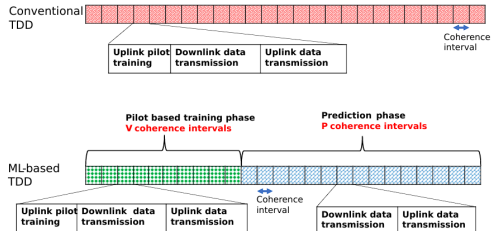
During the pilot training phase, the vectorized signal received at the BS from the k^{th} user across T time slots is modeled as:

$$\bar{\mathbf{y}}_k[l] = \mathbf{\Theta}_k[l] \bar{\mathbf{f}}_k[l] + \bar{\mathbf{v}}_k[l]$$

By employing a Least Squares (LS) estimation approach, the direct and cascaded RIS channels are extracted:

$$\hat{\mathbf{f}}_k[l] = (\mathbf{\Theta}_k[l]^H \mathbf{\Theta}_k[l])^{-1} \mathbf{\Theta}_k[l]^H \bar{\mathbf{y}}_k[l]$$

This extracted initial CSI acts as the critical foundational input for the machine learning prediction phase.



Channel Aging Model & Autoregressive (AR) Prediction

The Channel Memory

We model the channel "memory" using **Jakes' Autocorrelation Function** (J_0). This quantifies exactly how much past CSI values influence the present.

Q-th Order AR Model

Predicts a channel using its own past values:

$$h_k[l] = - \sum_{q=1}^Q a_{k,q} h_k[l-q] + \omega_k[l]$$

- $\mathbf{h}_k$: Target channel to predict.
- Q : Order (Memory). e.g., $Q = 24$ uses 24

Levinson-Durbin Recursion

The AR coefficients are evaluated via:

$$\mathbf{a} = -\mathbf{R}^{-1}\mathbf{w}$$

Where $\mathbf{R}$ is the correlation matrix of past CSI samples:

$$\mathbf{R} = \begin{bmatrix} R[0] & R[-1] & \cdots & R[-Q+1] \\ R[1] & R[0] & \cdots & R[-Q+2] \\ \vdots & \vdots & \ddots & \vdots \\ R[Q-1] & R[Q-2] & \cdots & R[0] \end{bmatrix}$$

$$\mathbf{w} = [R[1], R[2], \dots, R[Q]]^T$$

Overcoming Matrix Singularity: Diagonal Loading

The Singularity Problem

- If the user is moving slowly, the channel doesn't change much.
- This means $R[0]$, $R[1]$, and $R[2]$ are almost identical (e.g., 1.0, 0.999, 0.998).
- Consequently, the rows of the $\mathbf{R}$ matrix become **linearly dependent**, which makes the matrix singular and crashes the calculation.

The Solution: Diagonal Loading

- We "fix" the matrix by adding a tiny positive constant (ϵ) *only* to the main diagonal.
- This artificially "pushes" the eigenvalues of the matrix away from zero, making it much easier and more stable for the computer to invert.

Diagonal Loading Equation

$$\hat{R}[q] = \begin{cases} R[0] + \epsilon, & q = 0 \\ R[q], & q = 1, 2, \dots, Q. \end{cases}$$

Proposed Solution: The CNN-AR Framework

Deep Learning Integration

To bypass the limitations of the standard AR model, we replaced real-time mathematical weight calculations with a Convolutional Neural Network (CNN). This network is specifically designed to automatically learn complex, non-linear aging patterns from the cascaded channel.

The Prediction Workflow

The estimated complex channel vectors from V coherence intervals are first transformed into a real-valued matrix containing their isolated real and imaginary components. The CNN treats these past complex CSI samples as temporal "images", identifies the specific channel variation pattern, and outputs pre-computed AR coefficients in arbitrary order.

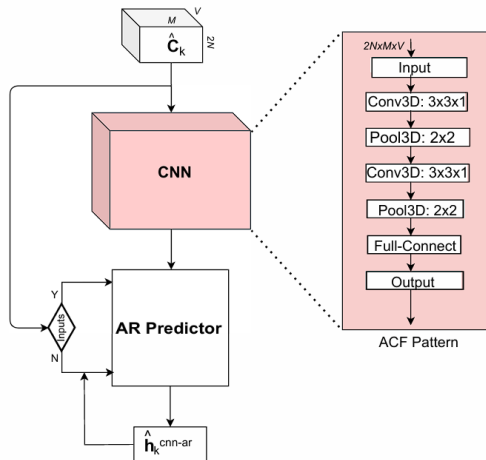
CNN Architecture & Data Pipeline

Network Structure

The model deploys Conv3D ($3 \times 3 \times 1$) layers dedicated to spatial-temporal extraction, immediately followed by Pool3D (2×2) downsampling layers.

Activation & Output

We utilize the tanh activation function in the convolutional layers to handle complex data variations, and a sigmoid function in the fully connected layers. Two fully connected layers (consisting of 512 and 256 neurons) process the final data to output the distinct Autocorrelation Function (ACF) pattern.



Dataset, Training & Evaluation

Dataset

- Channel model: Jakes (Bessel function J_0)
- Synthetic simulation data
- Samples: $\sim 70,000$
- Each sample:
 - Input: $V = 50$ past CSI samples
 - Prediction: next $P = 1-20$ intervals
- Parameters:
 - $f_c = 3$ GHz
 - Velocity ≈ 18 km/h
 - Sampling time = 1 ms

Training Setup

- Optimizer: Adam
- Loss: Mean Squared Error (MSE)
- Supervised learning:
 - Input $\rightarrow$ CSI history
 - Target $\rightarrow$ true AR coefficients

Evaluation Metric

$$\text{NMSE} = \frac{\mathbb{E} [\|\hat{h} - h\|^2]}{\mathbb{E} [\|h\|^2]}$$

Key Insight

CNN learns global channel statistics, enabling more accurate and stable prediction than AR.

Pilot Overhead Section (Important for Results)

Conventional TDD Estimation

Standard approaches require pilot symbols in both the training (V) and prediction (P) phases:

$$P^{conv} = (MNK + NK)(V + P)$$

Proposed CNN-AR Approach

By replacing constant measurement with AI prediction, our framework only demands pilots during the initial V training intervals:

$$P^{cnn-ar} = (MNK + NK)V$$

The Impact: Eliminating the $(+P)$ requirement drastically cuts the total pilot overhead. This massive reduction frees up critical time slots, allowing the system to transmit more actual user data.

Results: Prediction Accuracy (NMSE)

Overcoming the AR Plateau

While we initially aimed for an NMSE floor of -25 dB, testing revealed that simply expanding the standard AR order ($Q = 8, 16, 24$) hardly improves prediction accuracy on its own. The baseline models quickly hit a performance ceiling.

CNN-AR Superiority

The CNN-AR model achieves significantly higher prediction accuracy across all prediction intervals. By learning the non-linear aging patterns instead of relying purely on real-time mathematical approximations, it bypasses the limitations of the standard AR filters.

Robustness Over Time

The model demonstrates extreme robustness to channel aging. The CNN-AR at the 20th prediction interval actually outperforms the standard AR predictor at the 10th interval across various user velocities.

Performance Comparison: NMSE vs Prediction Intervals

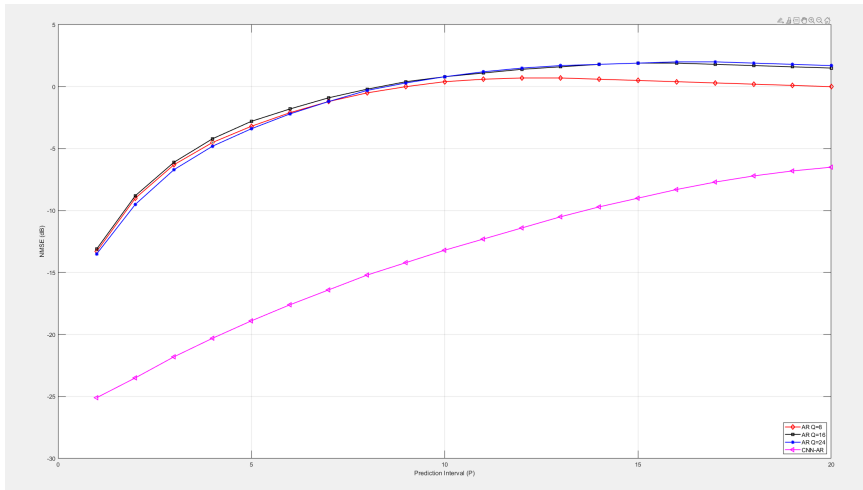


Fig: Prediction NMSE among the AR predictors ($Q = 8, 16, 24$) and CNN-AR over 20 prediction intervals.

Spectral Efficiency & Beamforming Optimization

Spectral Efficiency Definition

We jointly optimize the active beamforming vector ($\mathbf{u}_k[l]$) and passive reflection coefficients ($\theta[l]$):

$$\text{SE}_k[l] = \log_2 \left(1 + \frac{P_d}{\sigma_n^2} |(\mathbf{G}_k[l]\theta[l] + \mathbf{d}_k[l])^H \mathbf{u}_k[l]|^2 \right)$$

Optimization Problem (P1)

This forms a non-convex problem to maximize the received signal power:

$$\begin{aligned} \max_{\theta_m[l], \mathbf{u}_k[l]} \quad & |(\mathbf{G}_k[l]\theta[l] + \mathbf{d}_k[l])^H \mathbf{u}_k[l]|^2 \\ \text{s.t.} \quad & \|\mathbf{u}_k[l]\|^2 \leq 1 \\ & \theta_m[l] \in [0, 2\pi), \quad \forall m = 1, \dots, M. \end{aligned}$$

Alternating Optimization

By keeping $\theta[l]$ fixed, the optimal beamforming vector $\mathbf{u}_k^*[l]$ is found:

$$\mathbf{u}_k^*[l] = \frac{(\mathbf{G}_k[l]\theta[l] + \mathbf{d}_k[l])^H}{\|\mathbf{G}_k[l]\theta[l] + \mathbf{d}_k[l]\|}$$

Equivalent Problem (P2)

Substituting $\mathbf{u}_k^*[l]$ simplifies (P1) into (P2), solved using the **Semidefinite Relaxation (SDR)** technique:

$$\begin{aligned} \max_{\theta_m[l]} \quad & \|\mathbf{G}_k[l]\theta[l] + \mathbf{d}_k[l]\|^2 \\ \text{s.t.} \quad & 0 \leq \theta_m[l] \leq 2\pi, \quad \forall m = 1, \dots, M. \end{aligned}$$

Results: Spectral Efficiency (bps/Hz)

Impact on Data Rates

The ultimate utility metric for our system is the final boost in data rate achieved by precise RIS beamforming, calculated in bits per second per Hertz (bps/Hz).

Near-Perfect CSI Performance

The average spectral efficiency when channels are predicted using our CNN-AR model is remarkably similar to the theoretical “Perfect CSI” baseline. It significantly outperforms the standard AR estimator across all distances.

Optimal Placements

Efficiency naturally peaks when users are placed physically closer to either the BS or the RIS, which minimizes the “double fading” effect of the cascaded channel.

Performance Comparison: Spectral Efficiency

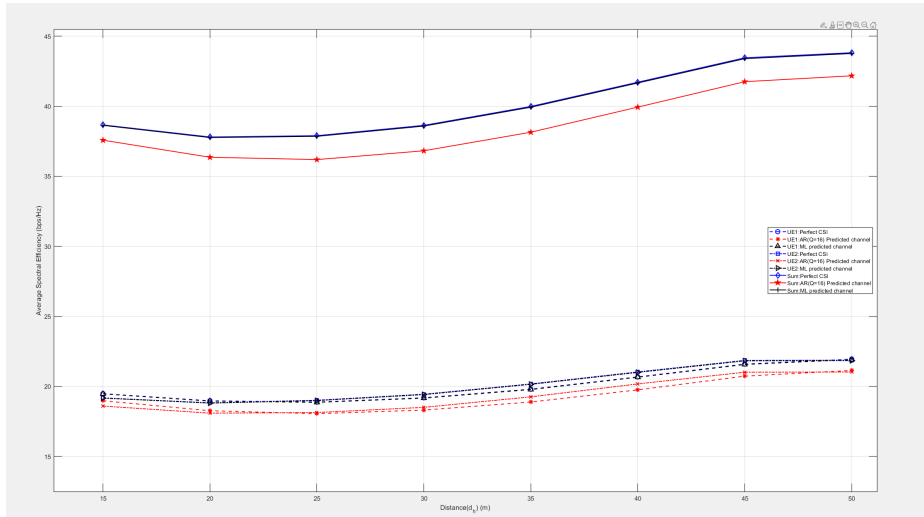


Fig: Average spectral efficiency versus varying user distances.

Conclusion

Successful Integration

We developed an efficient CNN-AR framework that successfully identifies complex channel aging patterns and provides enhanced channel estimates in correlated fast-fading environments.

High Precision & Scalability

The CNN-AR approach demonstrated high-precision estimation accuracy, bypassing the inherent limitations and frequent crashes of real-time mathematical approximations.

System-Wide Upgrades

Compared to traditional TDD methods, this integration achieves a demonstrably higher spectral efficiency while simultaneously operating on a fraction of the traditional pilot overhead.